

TÀI LIỆU ĐẶC TẢ YÊU CẦU PHÁT TRIỂN (SRS)

Hệ thống Quản lý Farmstay - Django & MySQL

Tài liệu tổng hợp Review 1, Review 2 và Review 3

REVIEW 1

1. Giới thiệu bài toán

Hiện nay, việc quản lý một farmstay thường gặp khó khăn trong việc đồng bộ hóa dữ liệu đặt phòng, quản lý dịch vụ (ăn uống, thuê xe) và kiểm soát luồng thanh toán. Quy trình thủ công dễ dẫn đến sai sót và nguy cơ "double-booking".

Bài toán đặt ra là xây dựng một hệ thống website quản lý Farmstay tập trung sử dụng Django Framework (Python) kết hợp cơ sở dữ liệu quan hệ MySQL (triển khai qua XAMPP cho môi trường phát triển). Hệ thống được tinh gọn với hai vai trò cốt lõi: Khách hàng (tự thao tác đặt phòng, thanh toán) và Admin (quản trị toàn bộ dữ liệu, dịch vụ, trạng thái đơn hàng và kiểm duyệt đánh giá).

2. Functional Requirements + Non-functional Requirements

Functional Requirements (Yêu cầu chức năng):

- Customer Portal (Khách hàng):
 - Đăng ký / Đăng nhập tài khoản.
 - Xem chỗ trống theo khoảng thời gian check-in/check-out.
 - Đặt phòng và thanh toán trực tuyến ngay lập tức qua cổng thanh toán.
 - Nhận email tự động xác nhận mã đặt phòng và số tiền đã thanh toán.
 - Đặt trước Menu thức ăn/nước uống (gắn liền với mã đặt phòng).
 - Thuê xe máy theo ngày.
 - Xem thư viện hình ảnh và gửi đánh giá (Review) sau khi hoàn thành chuyến đi.
- Admin Dashboard (Quản trị viên):
 - Tổng quan: Thống kê doanh thu, hiển thị booking chờ xử lý, trạng thái các phòng.
 - Quản lý Phòng & Dịch vụ: CRUD phòng, xe máy, menu nhà hàng.
 - Quản lý Booking: Theo dõi và cập nhật trạng thái đơn.
 - Quản lý Tài khoản: Cấp lại mật khẩu, toggle (On/Off) để khóa/mở khóa tài khoản khách hàng.

- Quản lý Đánh giá (Reviews): Xem danh sách các bài review và có quyền Xóa (Ẩn) các bài đánh giá vi phạm hoặc spam.

Non-functional Requirements (Yêu cầu phi chức năng):

- Xử lý đồng thời (Concurrency): Bắt buộc áp dụng cơ chế Database Locking (ví dụ: `select_for_update()` trong Django ORM) để khóa dòng dữ liệu (row-level lock) trong MySQL khi có giao dịch thanh toán, ngăn chặn triệt để double-booking.
- Bảo mật: Hash mật khẩu người dùng (PBKDF2/Argon2). Xác thực giao dịch API thanh toán qua chữ ký điện tử.
- Kiến trúc: Triển khai theo mô hình MVT của Django hoặc dùng Django REST Framework (DRF).

3. System required (Cấu hình phần cứng & Phần mềm)

- Stack Công nghệ: Backend: Django (Python). Database: MySQL (sử dụng Control Panel của XAMPP để chạy local service). Thư viện kết nối: mysqlclient hoặc PyMySQL.
- Môi trường Local: RAM tối thiểu 8GB.
- Môi trường Production: OS: Ubuntu 22.04 LTS; CPU: 2-4 Cores; RAM: 4GB-8GB; Storage: 50GB SSD. Triển khai MySQL Server độc lập (thay thế XAMPP).

4. Tasksheet 1

- Task 1.1: Chốt danh sách tính năng (2 Roles, CRUD cơ bản, có kiểm duyệt Review).
- Task 1.2: Khởi tạo project Django, cài đặt driver kết nối và thiết lập database MySQL (XAMPP) trong settings.py.
- Task 1.3: Phác thảo luồng xử lý thanh toán và cơ chế gửi email bất đồng bộ.
- Task 1.4: Thiết kế giao diện (UI/UX Mockups).

REVIEW 2

1. Use case

- Khách hàng: Tra cứu phòng trống, Đặt phòng & Thanh toán trực tuyến, Xem lịch sử booking, Đặt dịch vụ, Viết đánh giá.
- Admin: Quản lý CRUD (Phòng/Xe/Menu), Xem báo cáo, Cập nhật trạng thái đơn, Quản lý tài khoản, Xóa (Kiểm duyệt) bài đánh giá.

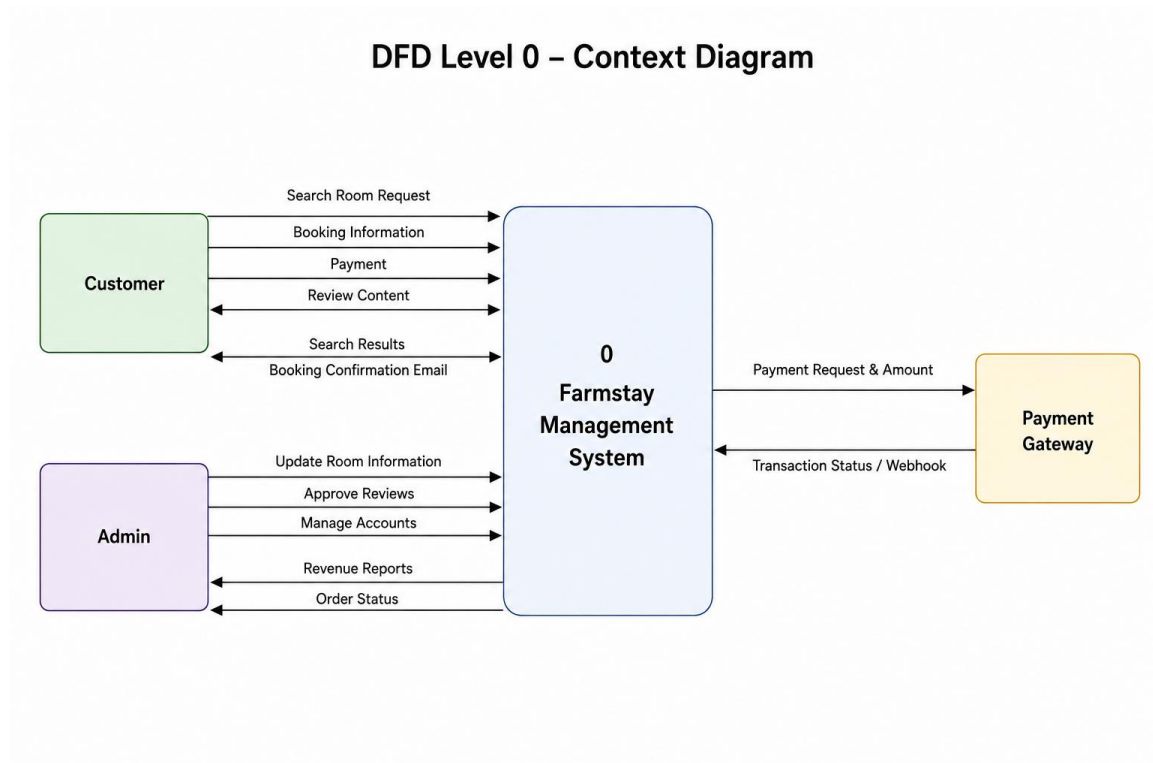
2. Sequence (Luồng Đặt phòng & Thanh toán)

1. Customer yêu cầu tra cứu phòng trống.
2. Django Backend truy vấn các bảng bookings và rooms trong MySQL để trả về phòng hợp lệ.
3. Customer nhập thông tin và nhấn "Thanh toán".
4. Django Backend tạo bản ghi booking với trạng thái PENDING_PAYMENT, gọi API cổng thanh toán.

5. Payment Gateway nhận thanh toán từ khách và bắn Webhook/Callback xác nhận về Django Backend.
6. Django Backend cập nhật trạng thái bản ghi thành PAID và kích hoạt luồng gửi Email xác nhận.

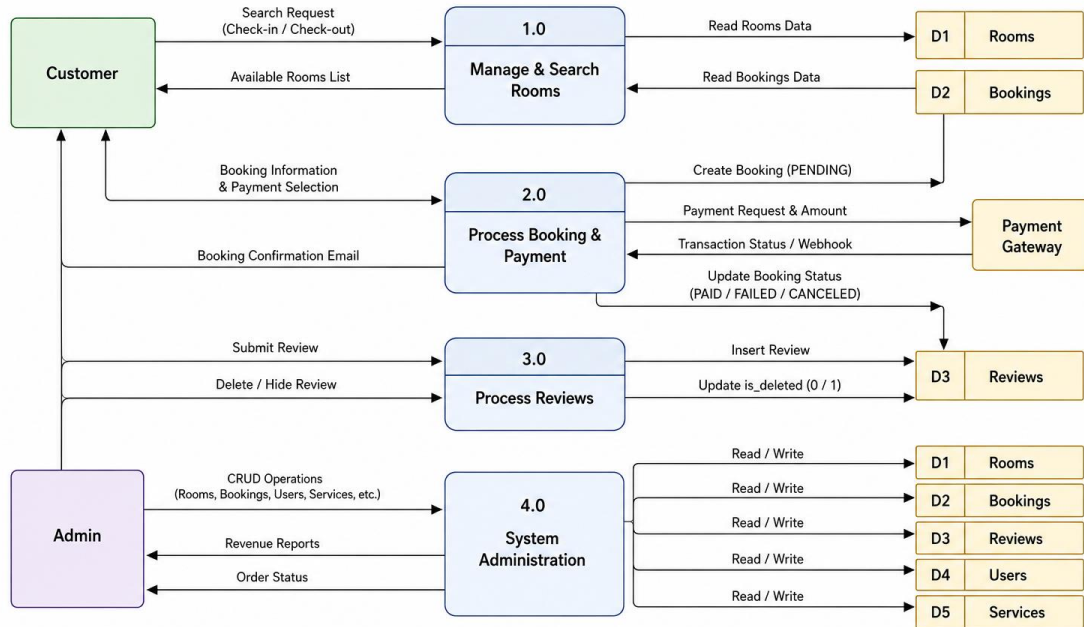
3. DFD (Data Flow Diagram)

- Level 0: Khách hàng tương tác với Hệ thống để gửi yêu cầu đặt chỗ/thanh toán, gửi review. Admin nhận báo cáo, cấu hình dữ liệu và kiểm duyệt nội dung (xóa review).



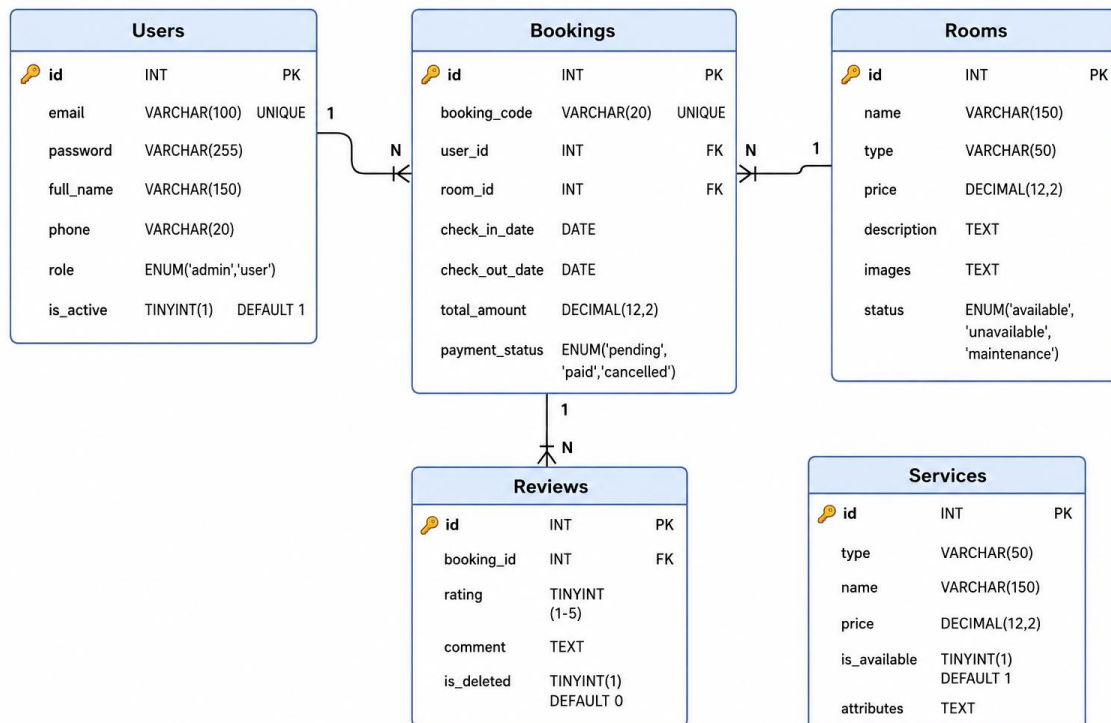
- Level 1: Tra cứu (Đọc từ MySQL) -> Đặt phòng & Giao dịch (Ghi bookings) -> Dịch vụ Email -> Hệ thống Review (Ghi reviews, Admin Update để ẩn).

DFD Level 1 – Detailed Breakdown



4. ERD (Mô hình Quan hệ Cơ sở dữ liệu)

Hệ thống sử dụng mô hình quan hệ chặt chẽ (Relational Model) với các Khóa chính (PK) và Khóa ngoại (FK) để ràng buộc dữ liệu: Users (1) - (N) Bookings, Rooms (1) - (N) Bookings, Bookings (1) - (N) Reviews, Bookings (1) - (N) MenuOrders.



5. Database Tables Description (MySQL Schema / Django Models)

- users: id (PK), email, password, full_name, phone, role, is_active
- rooms: id (PK), name, type, price, description, images (JSONField/Text), status
- bookings: id (PK), booking_code (Unique), user_id (FK), room_id (FK), check_in_date, check_out_date, total_amount, payment_status
- services: id (PK), type (FOOD/DRINK/MOTORBIKE), name, price, is_available, attributes (JSONField)
- reviews: id (PK), booking_id (FK), rating, comment, is_deleted (BooleanField - Default: False)

6. Task sheet 2

- Task 2.1: Định nghĩa Models trong Django và chạy lệnh makemigrations, migrate để tạo các bảng vật lý trong MySQL XAMPP.
- Task 2.2: Xây dựng API Controller cho CRUD (tận dụng Django Admin để quản lý Users, Rooms, Services và Reviews).
- Task 2.3: Logic tra cứu phòng trống bằng Django ORM (sử dụng exclude và filter để loại trừ phòng đã đặt).
- Task 2.4: Tích hợp SDK cổng thanh toán và hứng Callback (CSRF_exempt).
- Task 2.5: Setup SMTP và template email tự động.

REVIEW 3

1. Tasksheet review 3 (details to member)

- Member 1 (Backend Django & DB):
 - Xử lý rủi ro Concurrency: Implement và test kỹ lệnh `select_for_update()` khi thay đổi trạng thái thanh toán để tránh conflict dữ liệu.
 - Viết command `seed_data` tạo dữ liệu mẫu cho MySQL.
 - Viết filter API để Frontend chỉ load Review có `is_deleted = False`.
- Member 2 (Frontend/Django Templates):
 - Render dữ liệu lên giao diện.
 - Hoàn thiện luồng UX thanh toán (Loading, Success/Fail screens).
 - Hoàn thiện Dashboard Admin (Bổ sung tab Quản lý Review để Admin đọc và bấm Xóa/Ẩn).
- Member 3 (QA & Deployment):
 - Test E2E: Đặt phòng -> Thanh toán -> Viết Review -> Admin ẩn Review -> Frontend mất Review.
 - Deploy ứng dụng qua WSGI (Gunicorn/uWSGI) và Nginx.
 - Thiết lập và bảo mật MySQL Server cho môi trường Production (thay thế cho XAMPP local).